

Performance Engineering for AI Systems

Guojie Luo

- **Benchmarking & Performance Metrics**
- Profiling & Roofline Analysis
- Auto-Tuning & KernelGen
- End-to-End Optimization
- Lab: Performance Analysis in Practice

Why Do We Need Performance Metrics?

In High-Performance Computing (HPC) and system optimization,

we need to **quantify** program and hardware performance — different metrics reveal bottlenecks at different levels

Common Performance Metrics



FLOPS

Compute Capacity



Bandwidth

Bandwidth



Latency

Latency



Utilization

Resource Utilization

Roofline Model

Performance Ceiling Model

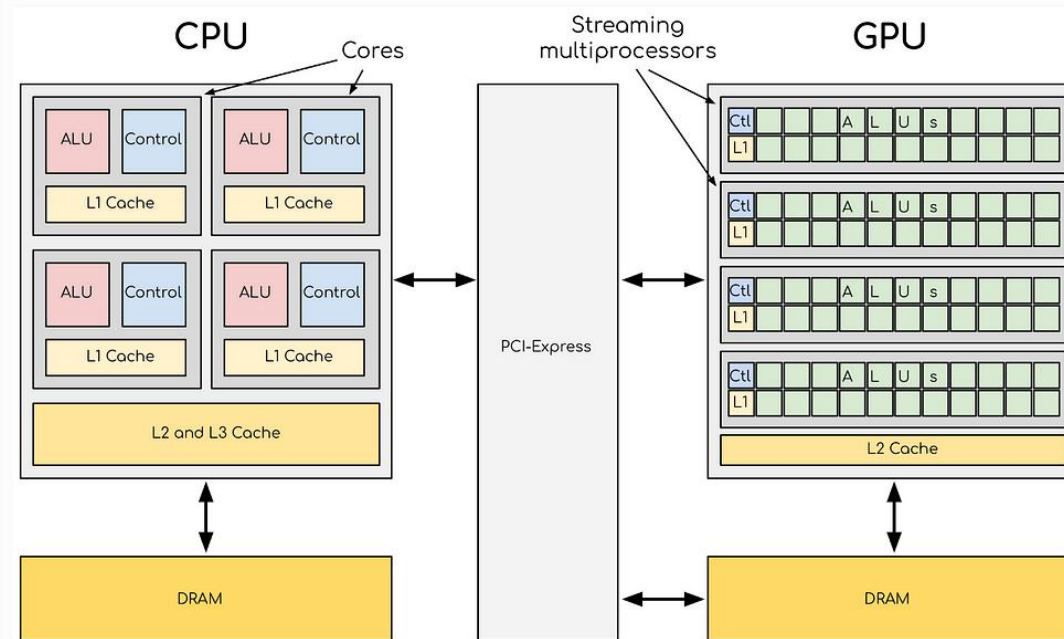


Image source: Medium - CPU vs GPU Architecture

Classification of Performance Metrics



Compute Capacity

Compute Bound



FLOPS

Measures the number of floating-point operations a processor can execute per second — the core metric for compute-intensive task performance.



Utilization

Reflects the actual usage efficiency of hardware resources, helping identify whether compute units are fully utilized.



Data Access Capability

Memory Bound



Bandwidth

Represents the amount of data transferable per unit time, determining throughput capacity for data-intensive applications.



Latency

Represents the time required to complete one data access, directly affecting program responsiveness and real-time performance.

Roofline Model used to analyze both bottleneck types, helping determine whether a program is compute-bound or memory-bound

What is FLOPS ?

FLOPS = Floating Point Operations Per Second

Assumed CPU Specs



Clock Frequency

3 GHz



FLOPs per Cycle

8 FLOP

Theoretical Performance Calculation

FLOPS = Frequency $\times$ FLOPs per Cycle

$= 3 \times 10^9 \times 8$

= 24 GFLOPS

Note: This is the **theoretical** peak; actual performance is typically lower

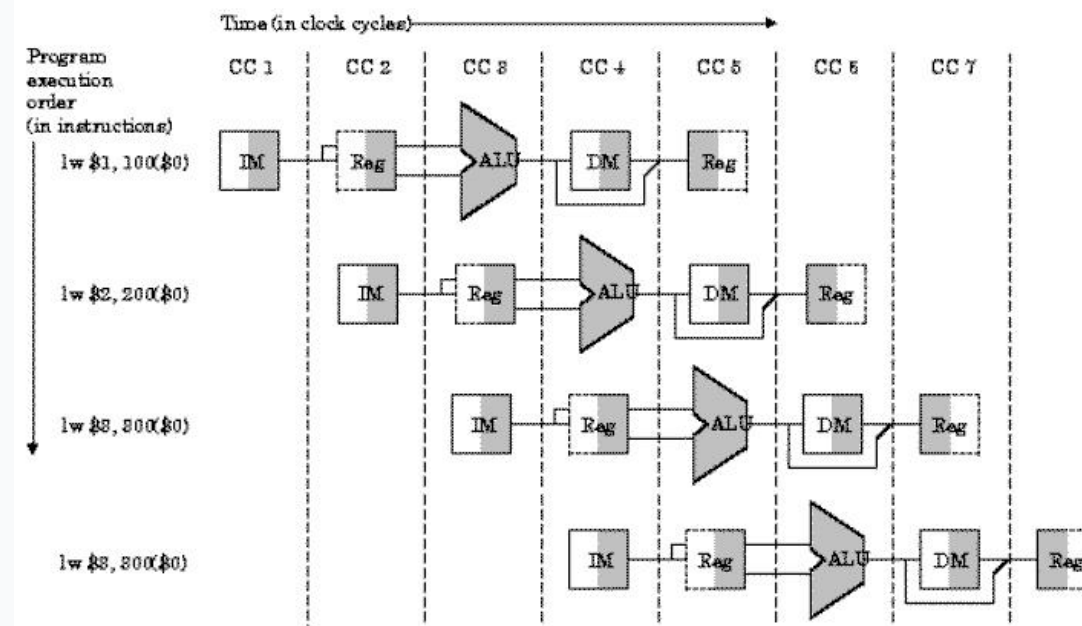


Image source: University of Florida - CPU Pipeline

Bandwidth

Bandwidth

Represents the amount of data transferable per unit time

Common Units

MB/s Megabytes/second

GB/s Gigabytes/second

TB/s Terabytes/second

Use Cases



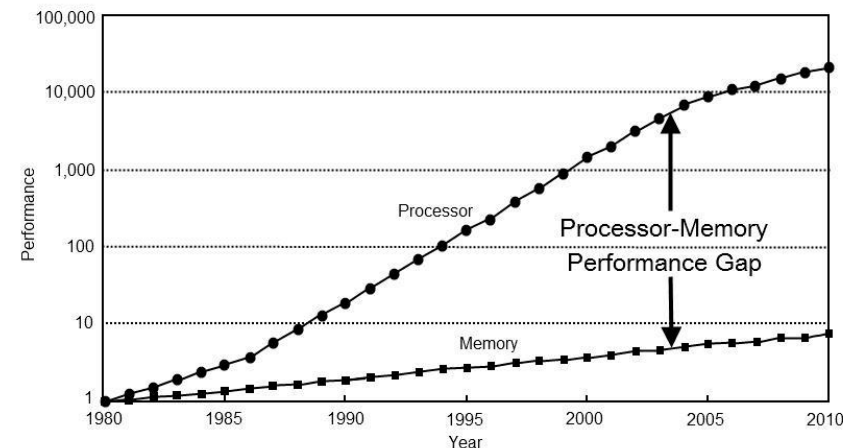
Memory
Bandwidth



Disk Bandwidth



Network
Bandwidth



MyComputer

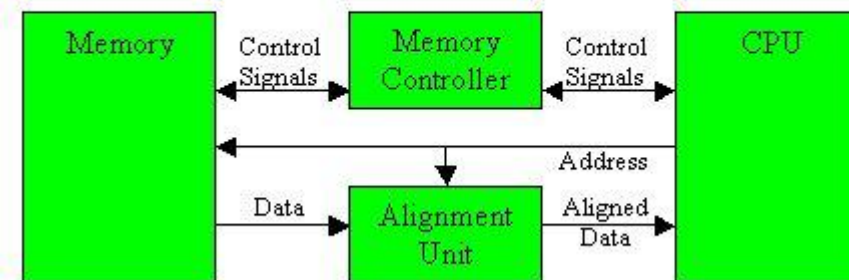


Image source: ForrestTheWoods - Memory Bandwidth,
University of Pittsburgh - Memory Controller

7 What is Latency?

Latency

Represents the time required to complete one operation

Typical Scenarios

Memory Access Latency

Network Latency

Disk Access Latency

Common Units

ns
Nanosecond
 10^{-9} second

μs
Microsecond
 10^{-6} second

ms
Millisecond
 10^{-3} second

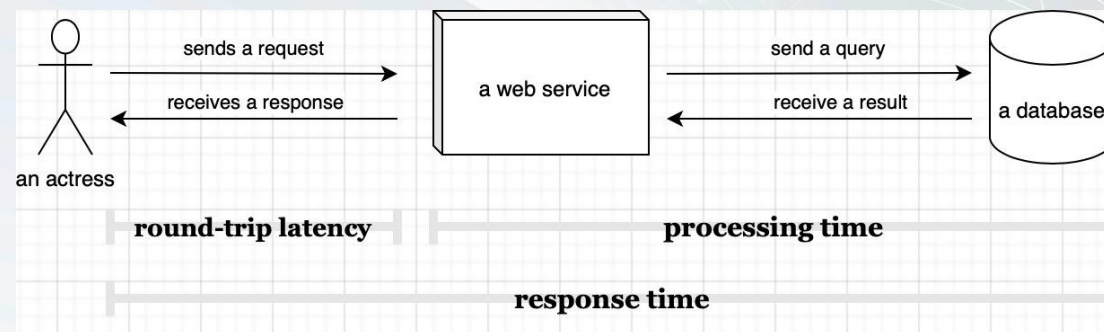


Image source: Scalable Developer - Latency Timeline

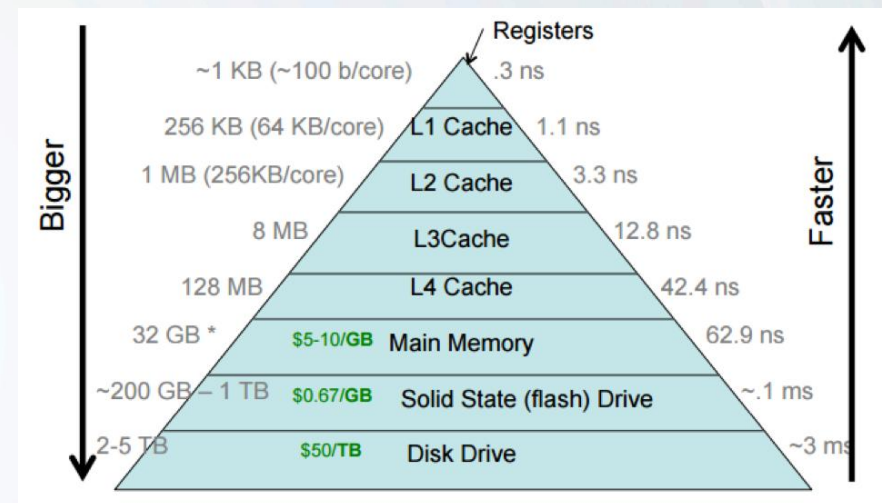


Image source: Brown University - Memory Hierarchy

Utilization

Utilization Actual usage level of hardware resources

Even with powerful hardware

if utilization is low → performance will still be poor

Formula

$$\text{Utilization} = \text{Actual Performance} / \text{Theoretical Peak}$$

Typically expressed as a percentage

Calculation Example

GPU Theoretical Performance

10 TFLOPS

Actual Program Performance

5 TFLOPS

Utilization

50%



Image source: Grafana - GPU Metrics Dashboard



Image source: Lambda Labs - GPU Monitoring Tools

MLPerf – Industry Standard for ML Performance



BAAI
智源研究院



北京大学
PEKING UNIVERSITY

What is MLPerf?

An industry-standard benchmark suite for machine learning performance

- ▶ Two main suites: MLPerf Training and MLPerf Inference

- ▶ Over 3,500 individual results from 100+ system configurations (as of 2021)

- ▶ Performance spans 5 orders of magnitude – from tiny embedded devices to cloud-scale datacenters

Why MLPerf matters?

- ▶ Provides fair, reproducible comparison across hardware and software

- ▶ Drives performance improvements – $1.3\times$ speedup in 16-chip results between v0.5 and v0.6 despite higher quality targets

- ▶ Growing adoption: 3 submitters (40+ results) → 14 submitters (500+ results) in early rounds

Key design principles:

- ▶ **Closed Division:** Fixed model for apples-to-apples hardware comparison

- ▶ **Open Division:** Any model allowed to encourage innovation

- ▶ **Time-to-train** as the primary metric for training (vs. throughput)

- ▶ **Semantic-level benchmarks:** Submitters reimplement models for their hardware

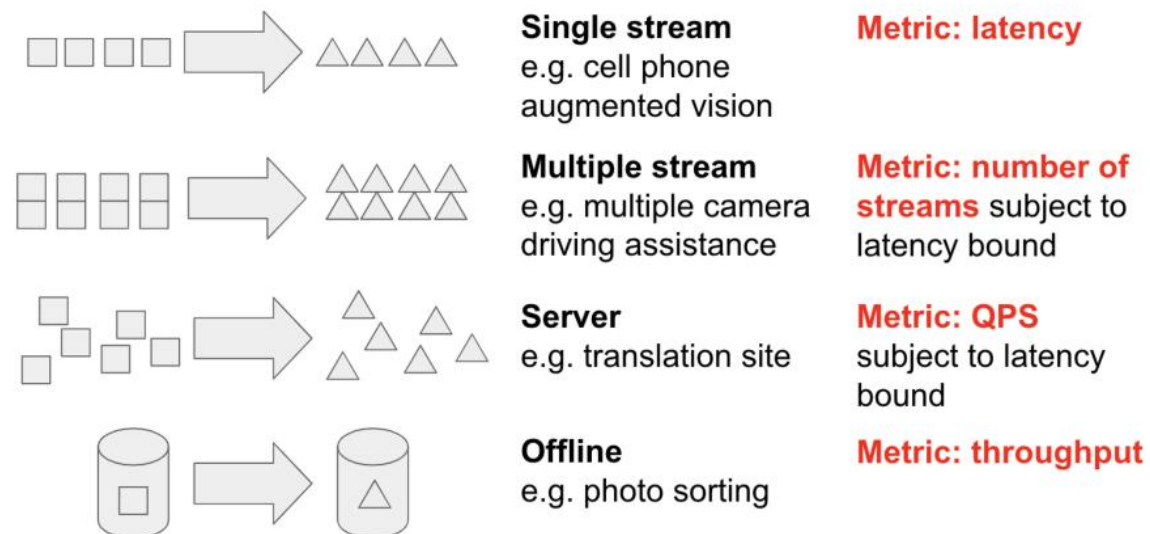
MLPerf Inference – Scenarios & Metrics

- **Quality targets:**

- Most tasks: within 1% of FP32 reference accuracy
- Medical imaging / recommendation: within 0.1% (real-world stringency)

- **Performance varies drastically across scenarios:**

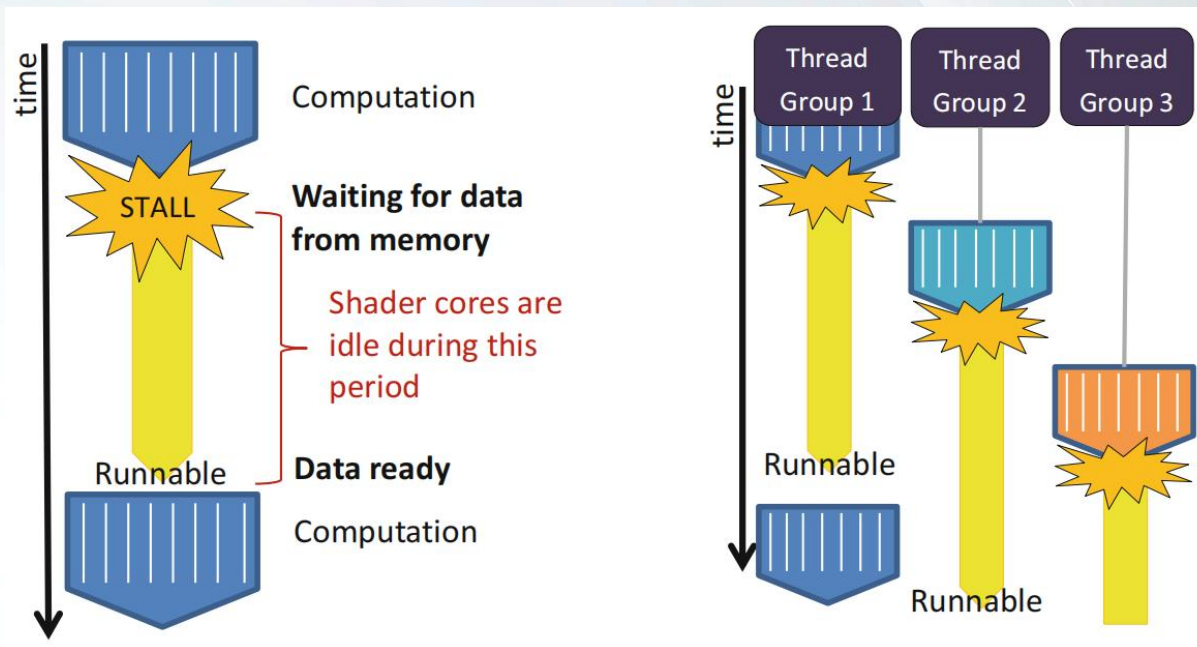
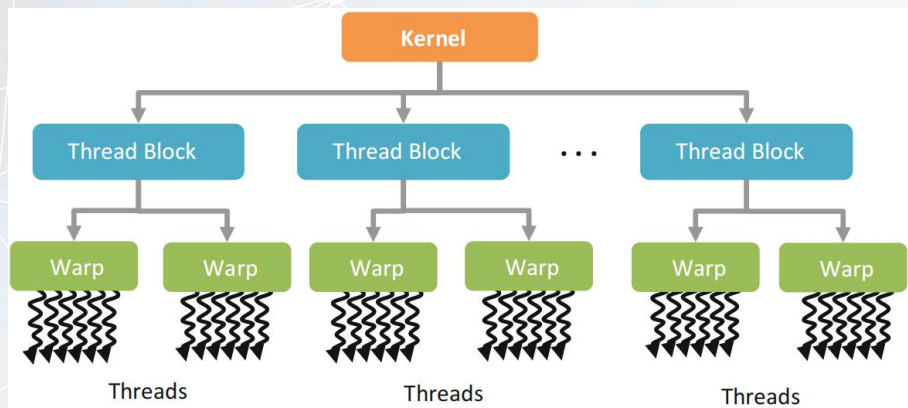
- Server vs Offline throughput ratio: up to $0.55\times$ (GNMT), $\sim 0.8\times$ on average
- Systems optimized for throughput may perform poorly under latency constraints



- Benchmarking & Performance Metrics
- **Profiling & Roofline Analysis**
- Auto-Tuning & KernelGen
- End-to-End Optimization
- Lab: Performance Analysis in Practice

Profiling answers: Where is the time spent inside the kernel?

- Execution model under GPU Architecture
 - GPU execution without/with batched thread groups
 - Single-instruction multiple threads (SIMT)
 - Hierarchical thread execution model

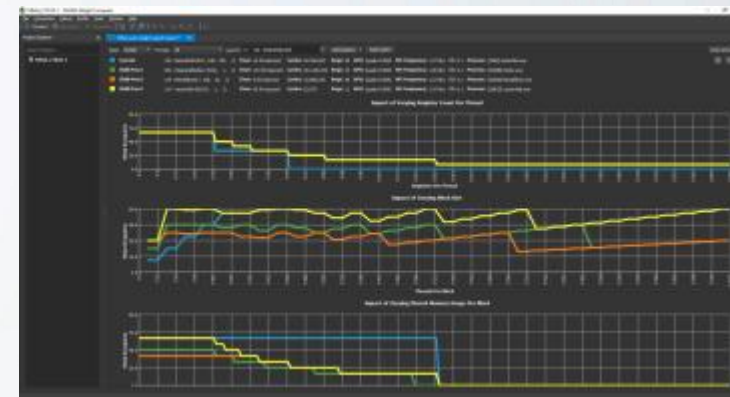


Occupancy – Keeping the GPU Busy

- Occupancy = Active Warps / Maximum Warps per SM
- Three limiting factors: Registers:
 - Each SM has limited registers (e.g., 16384)
 - Shared Memory: Limited capacity per SM
 - Thread Block Size: Hardware limit (NVIDIA: 1024, BI-V150: 4096)
- Example: Consider a GPU with 16384 registers per SM running a kernel that requires 35 registers per thread
 - Each SM can execute at most 468 threads
 - This affects the choice of block size
 - A block size of 512 is not possible
 - Only 1 block of 256 threads is allowed at a time, even though 212 more threads could be running
 - 3 blockss of 128 threads are allowed, providing 384 threads to be scheduled, etc.

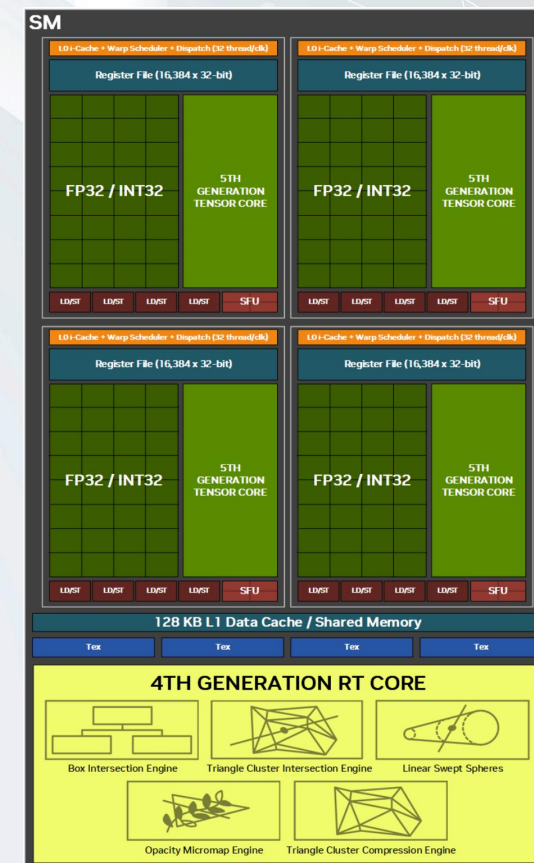
Occupancy – Limiting Factors

- The minimum of these three factors is what limits the active number of threads (or occupancy) of a compute unit
- The interactions between the factors are complex
 - The limiting factor may have either thread or wavefront granularity
 - Changing work group size may affect register or shared memory usage
 - Reducing any factor (such as register usage) slightly may allow another work group to be active
- The Nsight occupancy calculator from NVIDIA plots these factors visually allowing the tradeoffs to be visualized



Occupancy: Tradeoffs

- Shared memory and registers are persistent within SM when other warps execute
 - Allows for lower overhead context switch
- The number of active warps that can be supported per SM is limited
 - Decided by the shared memory per block and register usage per thread
- The number of active warps possible on an SM can be expressed using a metric called utilization
- Larger numbers of active warps allow for better latency hiding on both NV and AMD hardware



Profiling Tools on BI-V150 (Days Platform)



ixSMI: System-level monitoring

- ▶ `ixsmi`
- ▶ View GPU status (temp, power, memory)
- ▶ `ixsmi dmon`
- ▶ Real-time device monitoring
- ▶ `ixsmi pmon`
- ▶ Process monitoring

ixSYS: System-wide trace capture

- ▶ `ixsvs -t cuda.nvtx -o output.trace ./program`
- ▶ Captures CUDA API calls
- ▶ kernel execution
- ▶ Web-based visualization UI

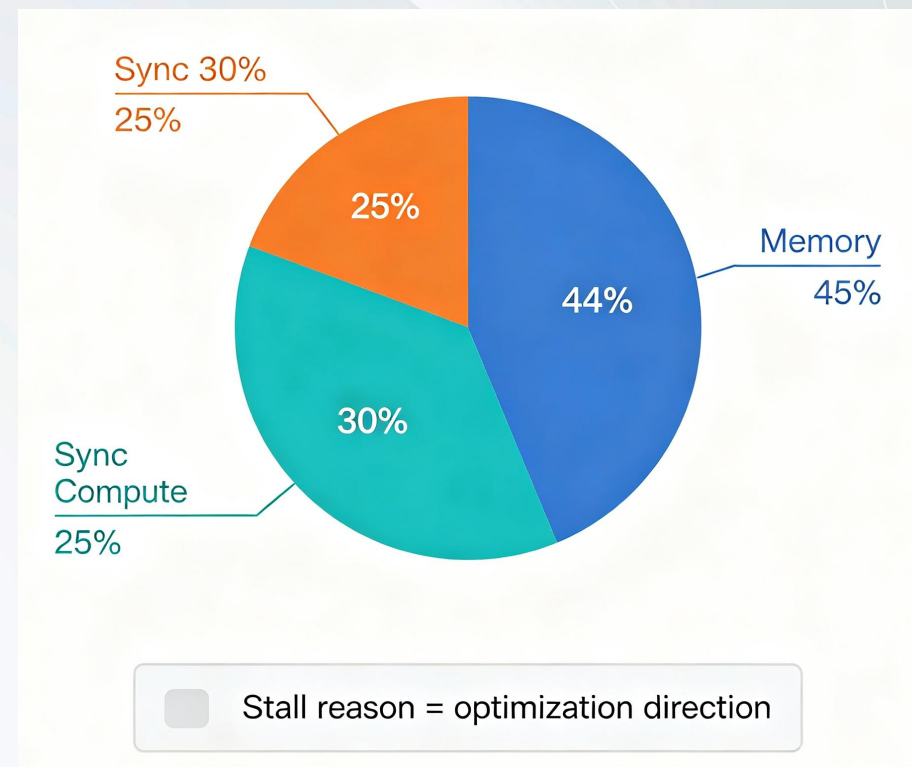
ixKN: Kernel-level performance analysis

- ▶ `ixkn-cli --set default ./program`
- ▶ Sections
 - ▶ Instruction
 - ▶ LaunchStats
 - ▶ Occupancy
 - ▶ WarpStateStats,
 - ▶ Memory

ixKN – A Deep Dive into Kernel Performance

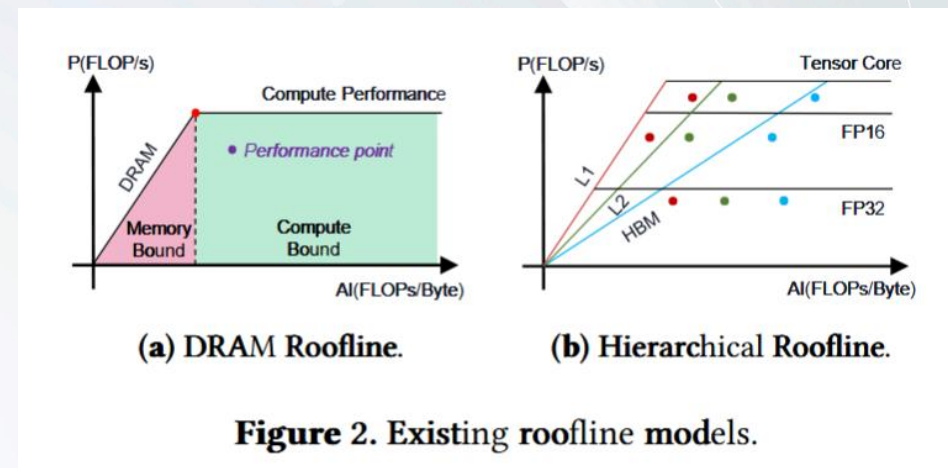
WarpStateStats example

Section	What It Tells You
LaunchStats	Grid/Block config, resource usage
Occupancy	Active warps vs. maximum, utilization rate
WarpStateStats	Why warps are stalling (memory, sync, compute)
Instruction	Instruction mix and execution counts
Memory	Shared memory bank conflicts, cache hit rates



The Roofline Model – Visualizing Performance Limits

- A visual performance analysis model that answers:
 - Is my program compute-bound or memory-bound?
 - How far am I from the hardware limit?
- The core insight: Performance is limited by two constraints:
 - Compute ceiling (horizontal line): $P \leq \pi$ (peak FLOP/s)
 - Bandwidth slope (diagonal line): $P \leq \beta \times I$ (bandwidth $\times$ arithmetic intensity)
- Ridge point: Intersection of horizontal and diagonal lines divides the space into two regions:
 - Left of ridge $\rightarrow$ Memory-bound (performance limited by bandwidth)
 - Right of ridge $\rightarrow$ Compute-bound (performance limited by arithmetic)



Beyond DRAM – Hierarchical & Component-Based Roofline

- Hierarchical Roofline:
 - CPUs/GPUs have multi-level caches (L1, L2, DRAM)
 - Any layer can be a bottleneck → build bandwidth ceilings for each layer
 - Also supports multiple precision/compute units (FP32, FP16, Tensor Core)
- Challenge: Roofline on DSAs (Ascend):
 - Ascend has unique compute units and memory transfers
 - Naive approach: build rooflines for every precision-transfer combination
 - For Ascend: 180 combinations → visualization complexity explodes!
- Component-Based Roofline Model:
 - Proposed in ASPLOS 2025 paper "Squeezing Operator Performance Potential for the Ascend Architecture"
 - Core idea: Abstract components to capture operator performance → identify bottleneck components
 - Instead of 180 combinations → focus on component-level bottlenecks
 - Results: 1.07× to 2.15× end-to-end speedups across 11 real-world tasks

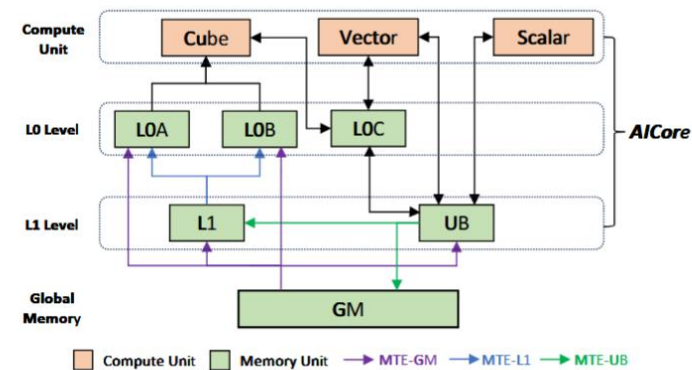
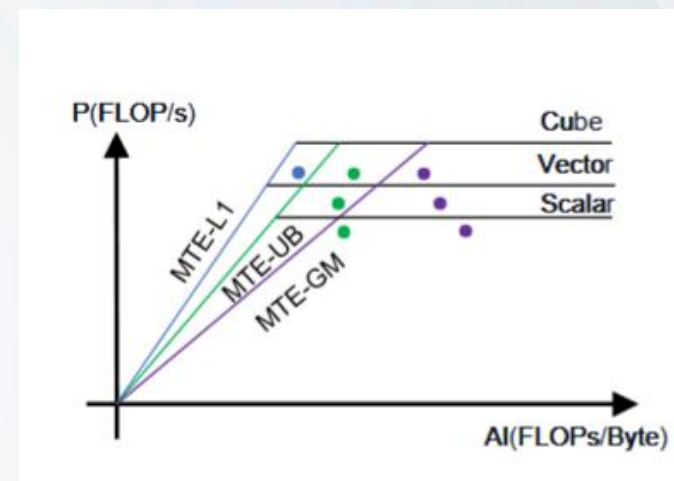


Figure 1. The architecture of AICore in Ascend.

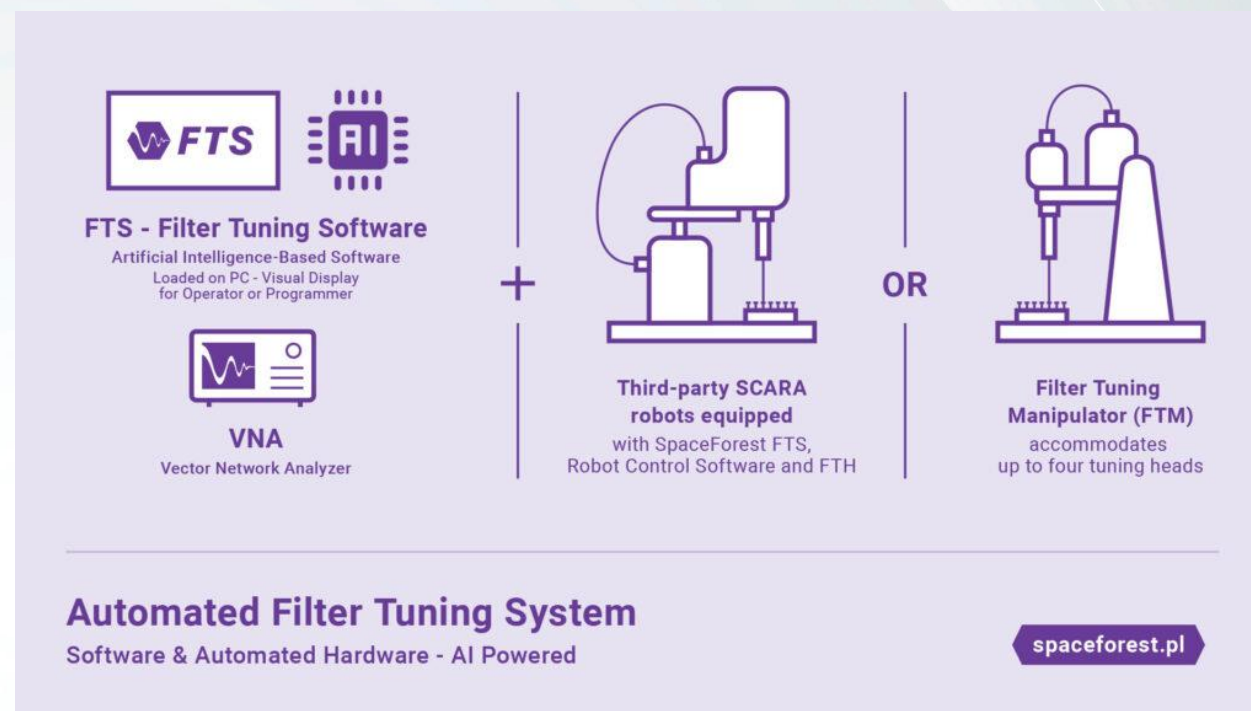
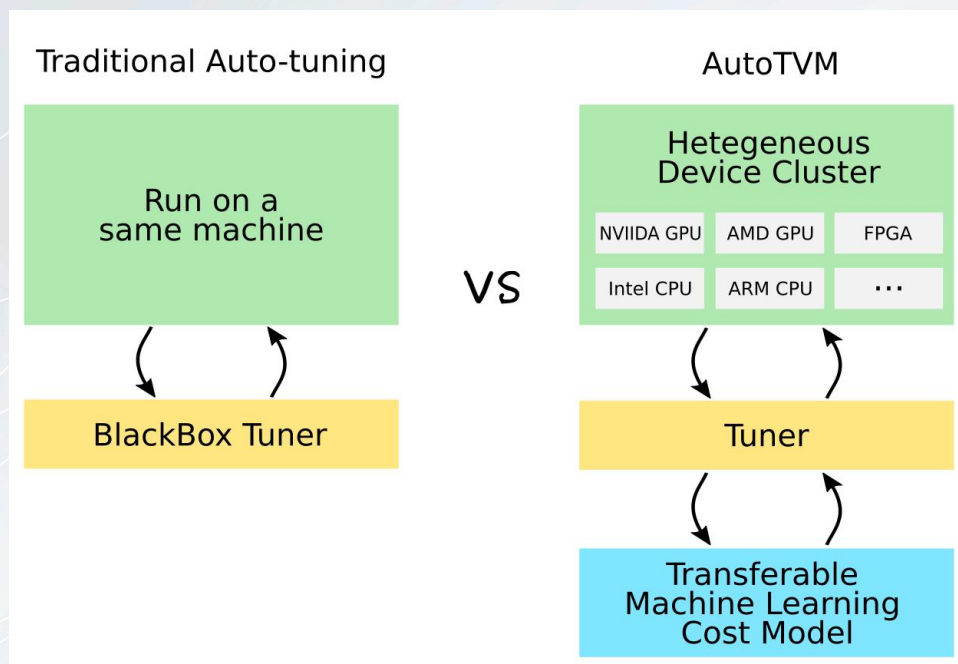


- Benchmarking & Performance Metrics
- Profiling & Roofline Analysis
- **Auto-Tuning & KernelGen**
- End-to-End Optimization
- Lab: Performance Analysis in Practice

Automatic Tuning

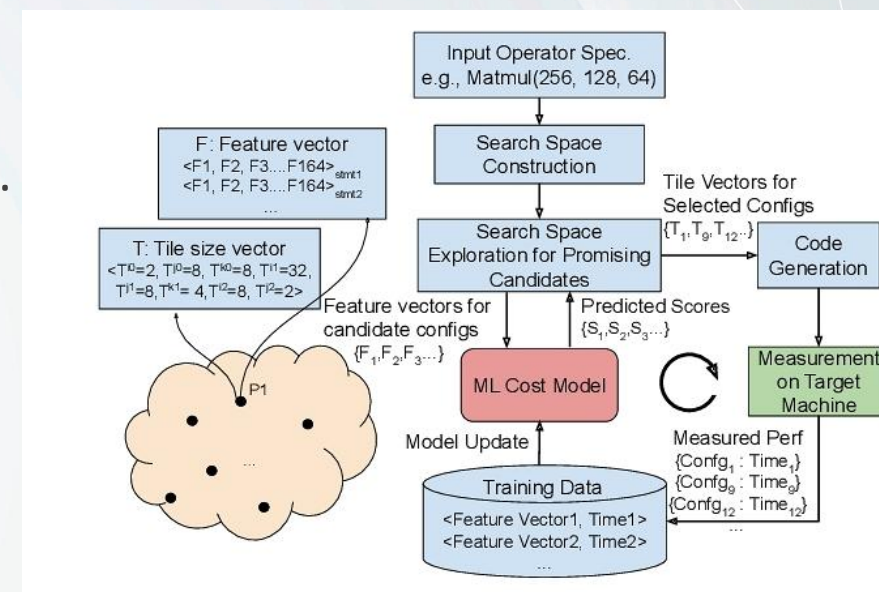


- Automatic Tuning Algorithm and Underlying Implementation
 - From Template Search to AI-Driven Operator Tuning



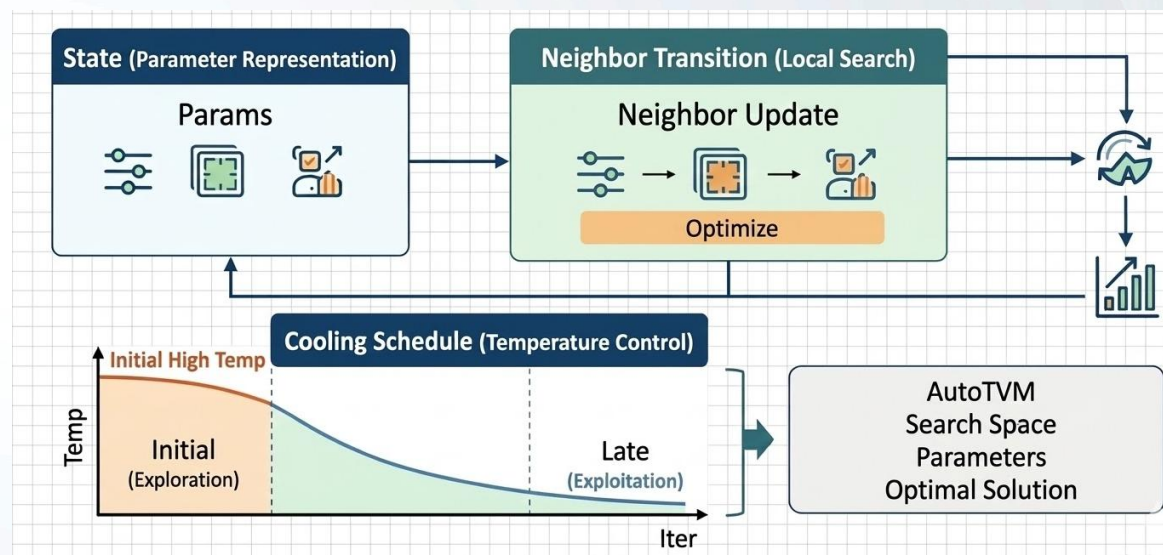
What is Kernel Auto-tuning?

- Automation of Code Generation:
 - Input: Mathematical calculation expressions (e.g., tensor multiplication, convolution).
 - Output: Low-level code optimized for specific hardware (e.g., CUDA, LLVM IR).
- Common Tuning Dimensions (Schedule Primitives):
 - Tiling: Improves cache hit rate.
 - Unrolling: Reduces branch overhead.
 - Vectorization: Utilizes SIMD instructions (e.g., AVX, TensorCore).
 - Binding: Block/Thread mapping in the GPU.



Application of Simulated Annealing

- State Representation: The current set of template parameters
- Neighbor Transition: Fine-tuning one of the parameters
- Cooling Schedule:
 - Initial High Temperature: High probability of accepting poor solutions (large-scale exploration).
 - Later Low Temperature: Almost only accepting good solutions (local convergence exploration).
- Early versions of AutoTVM widely used simulated annealing to find optimal parameters in the candidate space.



Advanced Exploration Algorithm: Genetic Algorithm

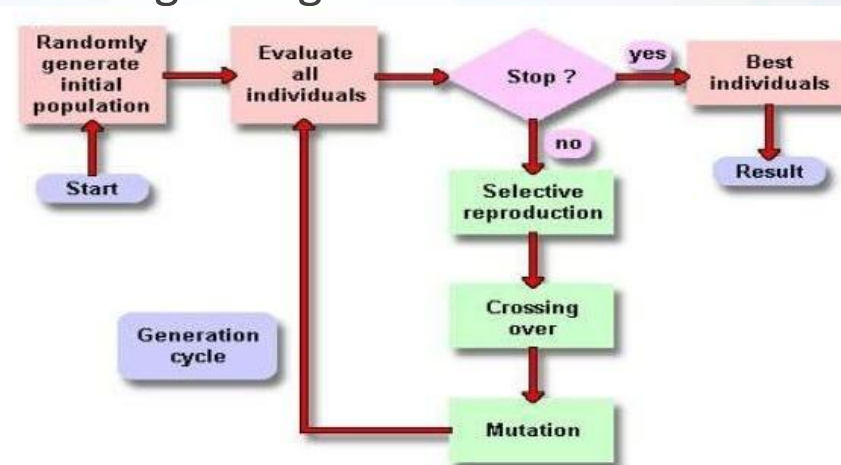
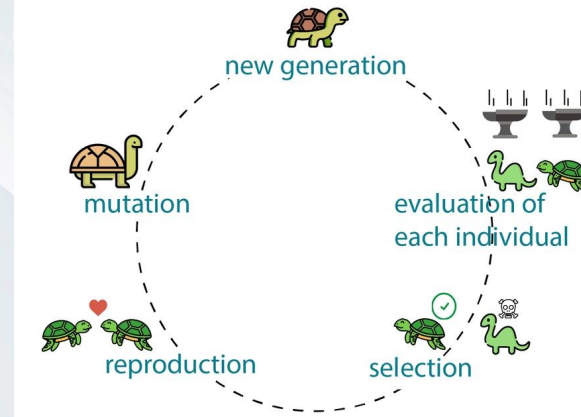


BAAI
智源研究院



北京大学
PEKING UNIVERSITY

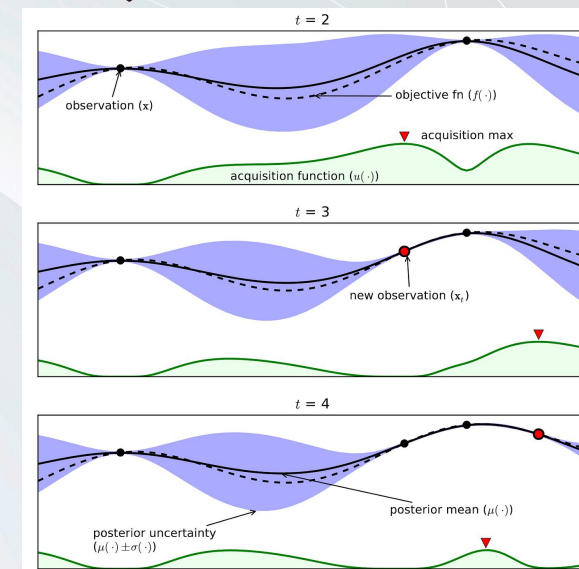
- Inspiration: Darwin's theory of evolution, "survival of the fittest."
- Core Algorithm Steps:
 - Population: Randomly generate a batch of initial configurations.
 - Fitness Evaluation: Evaluate performance using a cost model.
 - Crossover: Swap some parameters between two superior configurations to produce offspring.
 - Mutation: Randomly change some parameters (introducing new genes to prevent premature convergence).
 - Selection: Eliminate poorly performing configurations and retain the better ones for the next generation.



Advanced Techniques

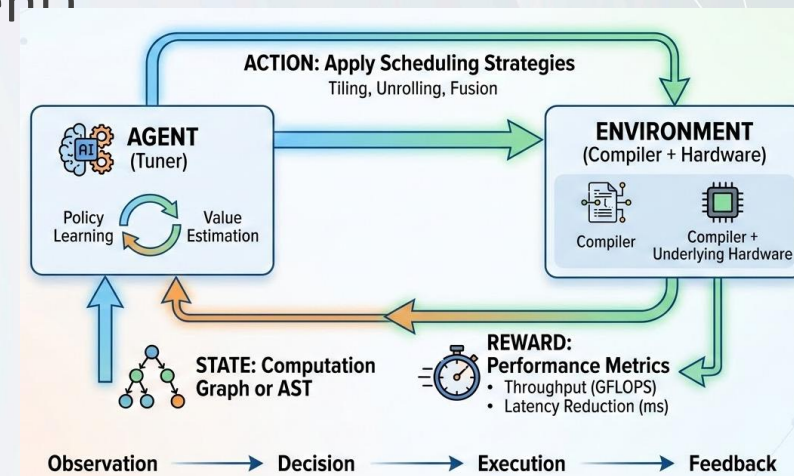
Bayesian

- Suitable Scenarios: Optimization of black-box functions where a single evaluation is extremely costly
- Core Components
 - ◆ Surrogate Model: Typically uses a Gaussian process
 - ◆ Acquisition Function: Often uses EI (Expected Improvement+)



Reinforcement Learning

- Tuning from a Reinforcement Learning Perspective (MDP - Markov Decision Process)



KernelGen background & workflow

KernelGen core objectives

Utilizing AI technology to automatically generate GPU kernel code, achieving ultimate performance optimization for target hardware and resolving the challenges of manual programming.

The industry faces challenges

The GPU programming model is complex, with large differences between architectures, lacking general optimization strategies, and having extremely high thresholds for parallel computing scheduling and memory management.

Core value of the project

Significantly lowering the threshold for GPU application development, unleashing the potential of hardware computing power, and improving the execution efficiency of computing tasks as well as resource utilization.

Step 1

Collect Kernel Information

User provides semantic operator definition to KernelGen

Step 2

Search Code Snippets

KernelGen searches similar code snippets as reference

Step 3

Generate Kernel Code and CUDA Baseline

KernelGen generates kernel code and CUDA baseline implementation

Step 4

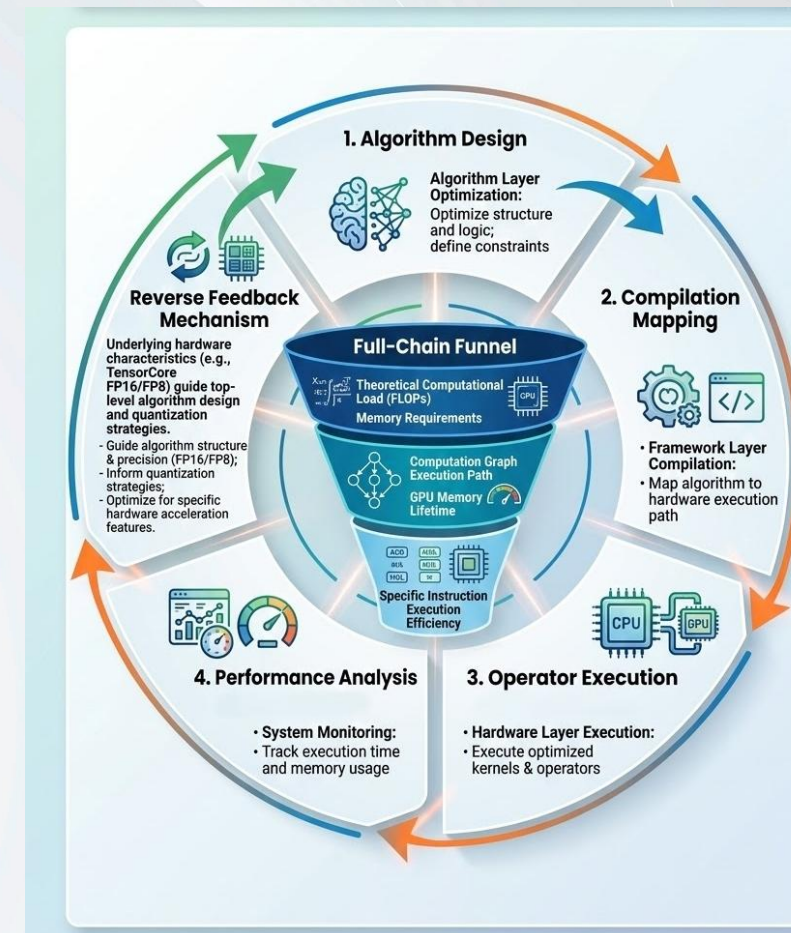
Test Kernel Against CUDA Baseline

KernelGen tests kernel against PyTorch baseline and outputs correctness and speedup results

- Benchmarking & Performance Metrics
- Profiling & Roofline Analysis
- Auto-Tuning & KernelGen
- **End-to-End Optimization**
- Lab: Performance Analysis in Practice

What is end-to-end collaborative optimization?

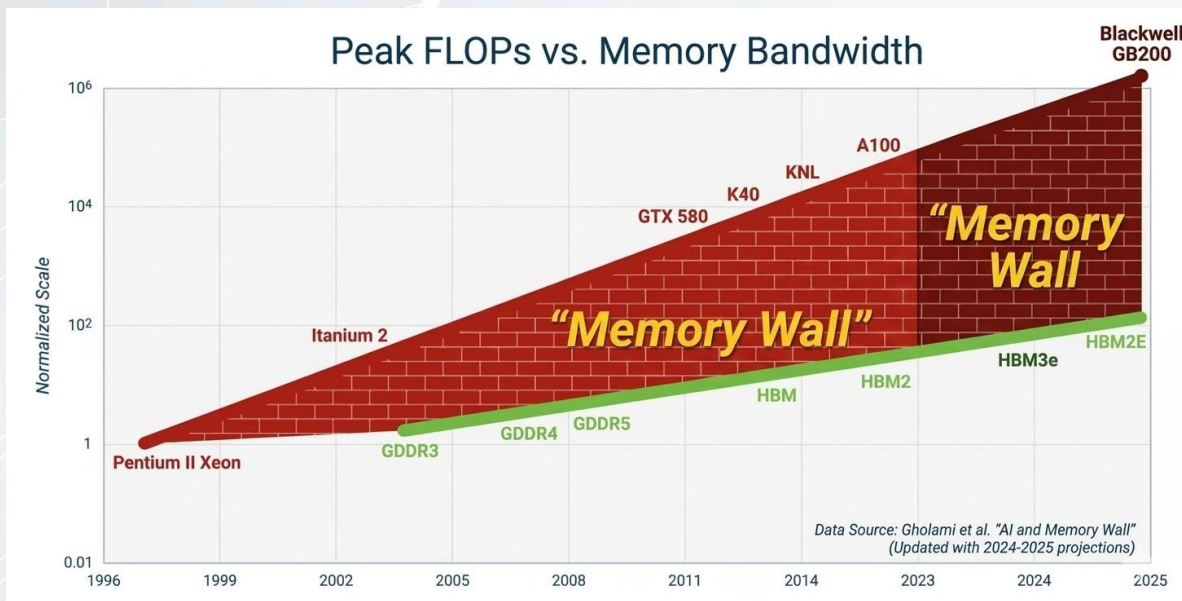
- Limitations of local optimization:
 - Amdahl's Law
- Full-chain funnel model:
 - Top layer (algorithm layer): theoretical computational load (FLOPs) and memory requirements.
 - Middle layer (compilation/framework layer): execution path of the computation graph and the lifetime of GPU memory.
 - Bottom layer (operator/hardware layer): execution efficiency of specific instructions.
- Reverse feedback mechanism
 - The characteristics of the underlying hardware (such as TensorCore's dedicated FP16/FP8) in turn guide the design and quantization strategy of the top-level algorithm.



The "Double Walls" of Large Model Inference

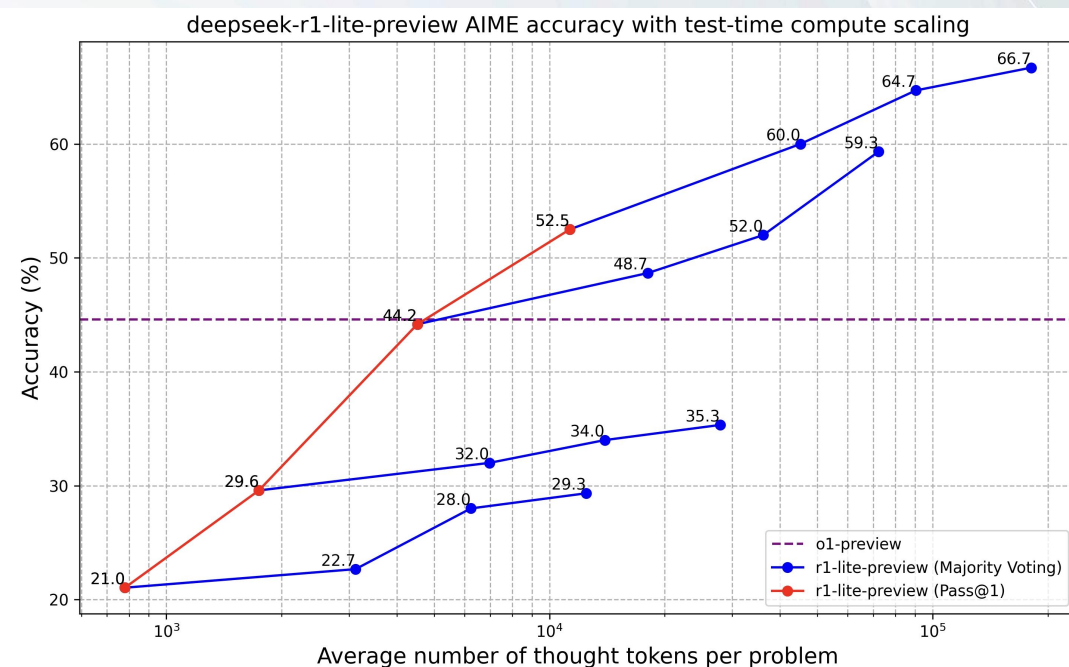
• Memory Wall:

- The decoding phase of LLM inference is primarily limited by memory access bandwidth.
- The massive key-value cache consumes a significant amount of GPU memory, limiting concurrent batch size.



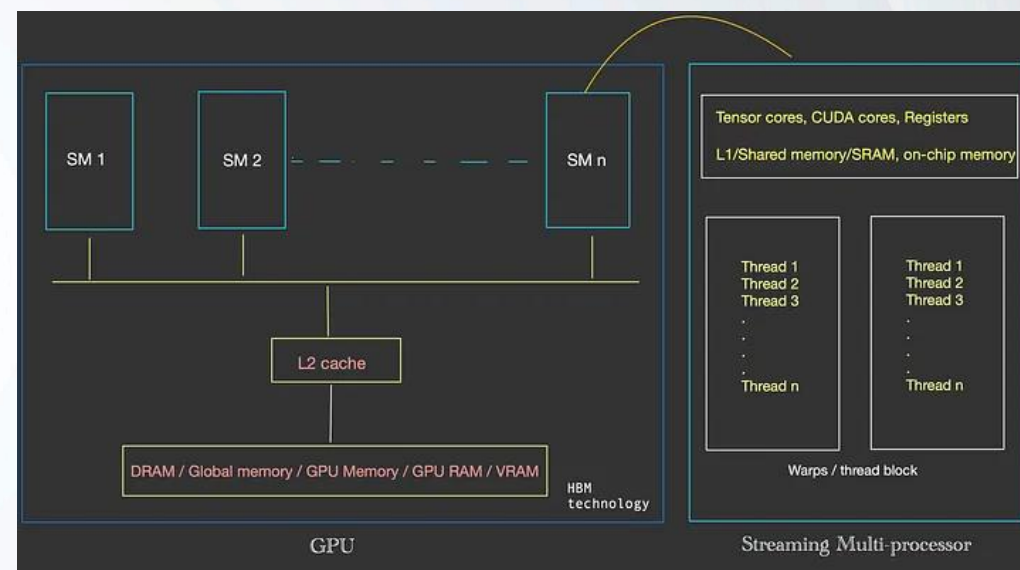
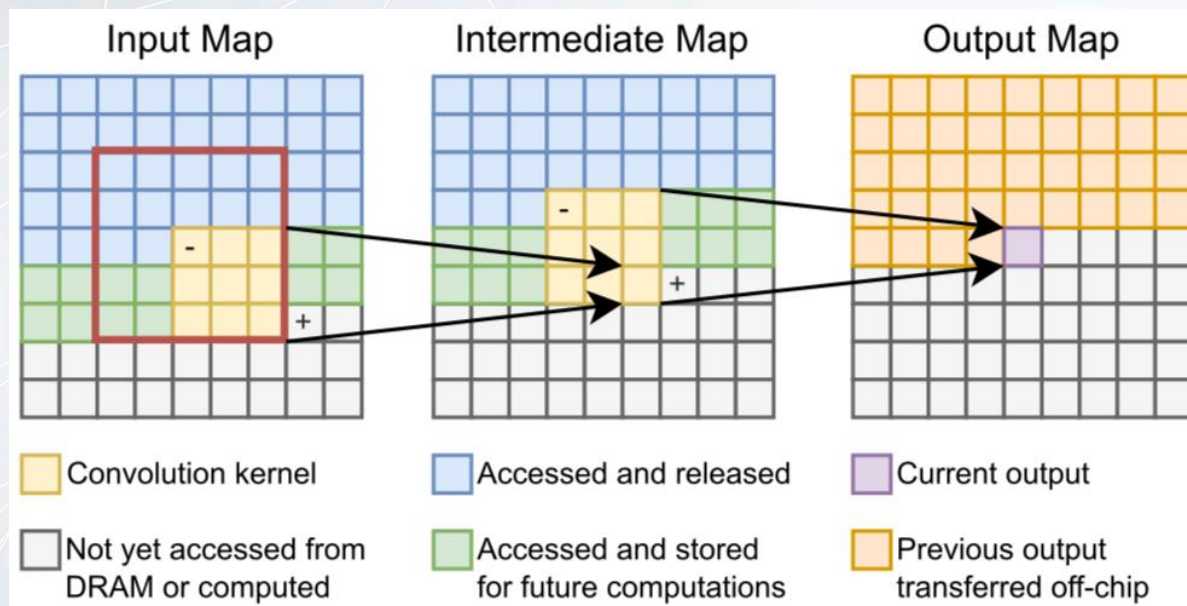
• Compute Wall:

- ◆ During the prefill phase, when handling long contexts, the computational cost of the attention mechanism increases quadratically with sequence length (compute-bound).



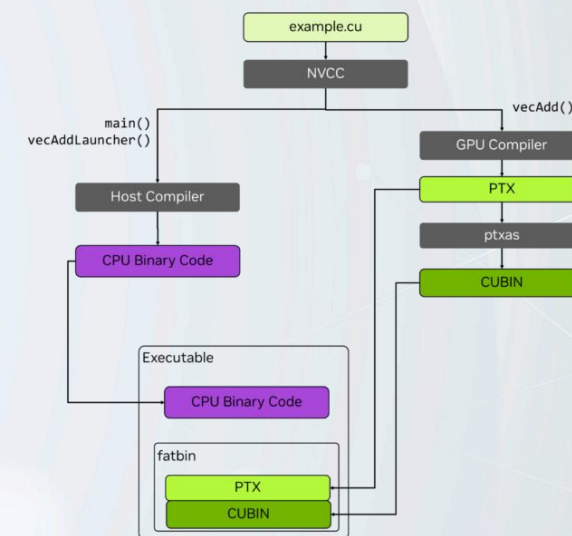
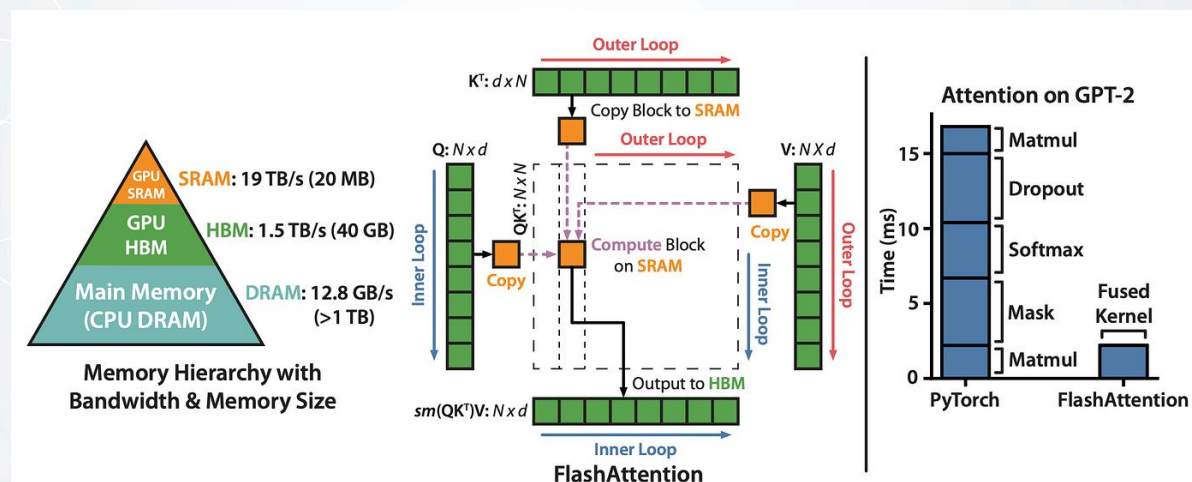
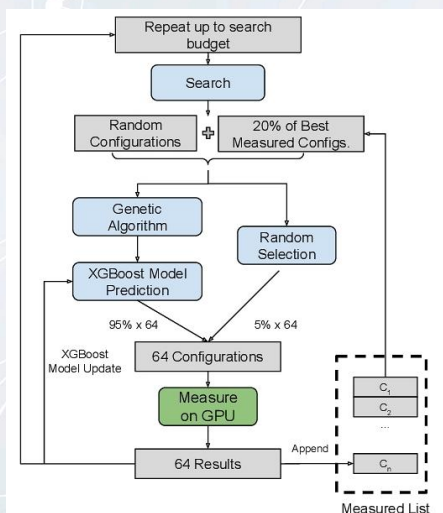
Compiler-Level Optimization Strategies

- Graph Optimization:
 - Operator Fusion: Merging multiple small operators (e.g., MatMul + Bias + ReLU) into a large kernel.
 - Benefits: Avoids repeated read/write operations to GPU memory (DRAM -> SRAM -> DRAM) for intermediate results; instead, it's completed directly in registers or shared memory, significantly improving bandwidth utilization.
 - Constant Folding & Dead Code Elimination



Operator Implementation

- Following up on the Automatic Tuning:
 - The optimal schedule found by AutoTVM or heuristic algorithms is compiled into the final hardware instructions.
- Core Optimization Techniques:
 - FlashAttention and its variants: Avoiding instantiating the complete Attention matrix into GPU memory through Tiling and Recomputation.
 - Extreme Assembly-Level Optimization: Squeezing out the last 5% of performance using PTX or a handwritten kernel tailored to a specific domestic chip microarchitecture.



The FlagOS Ecosystem

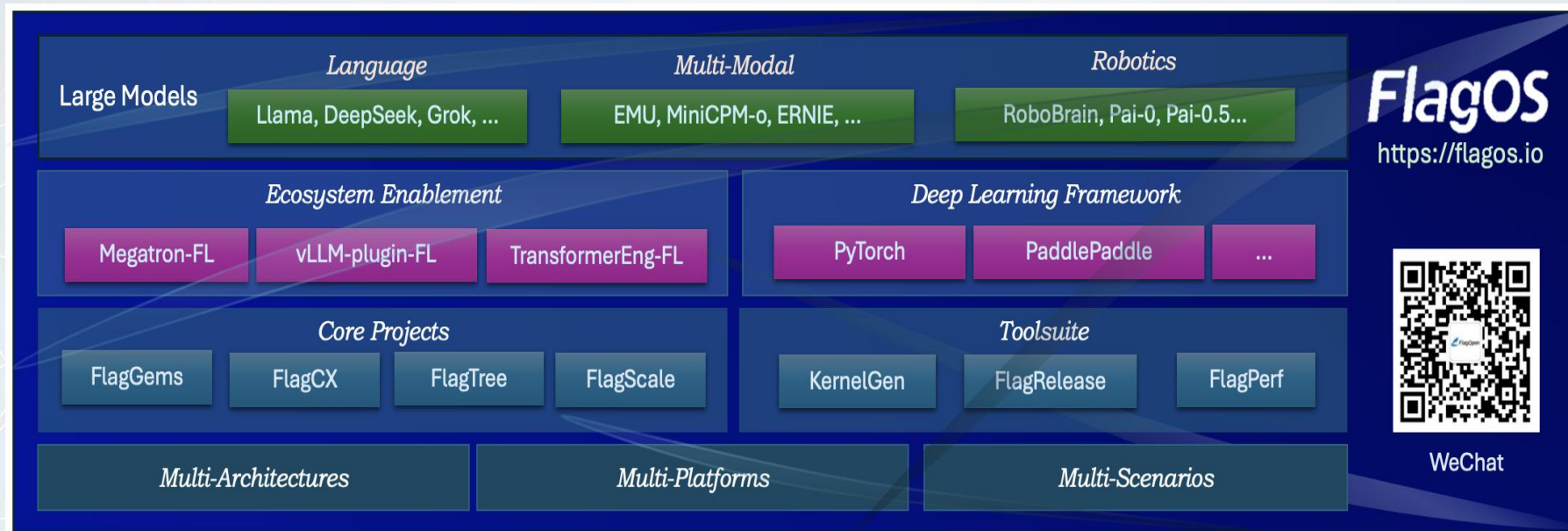


BAAI
智源研究院



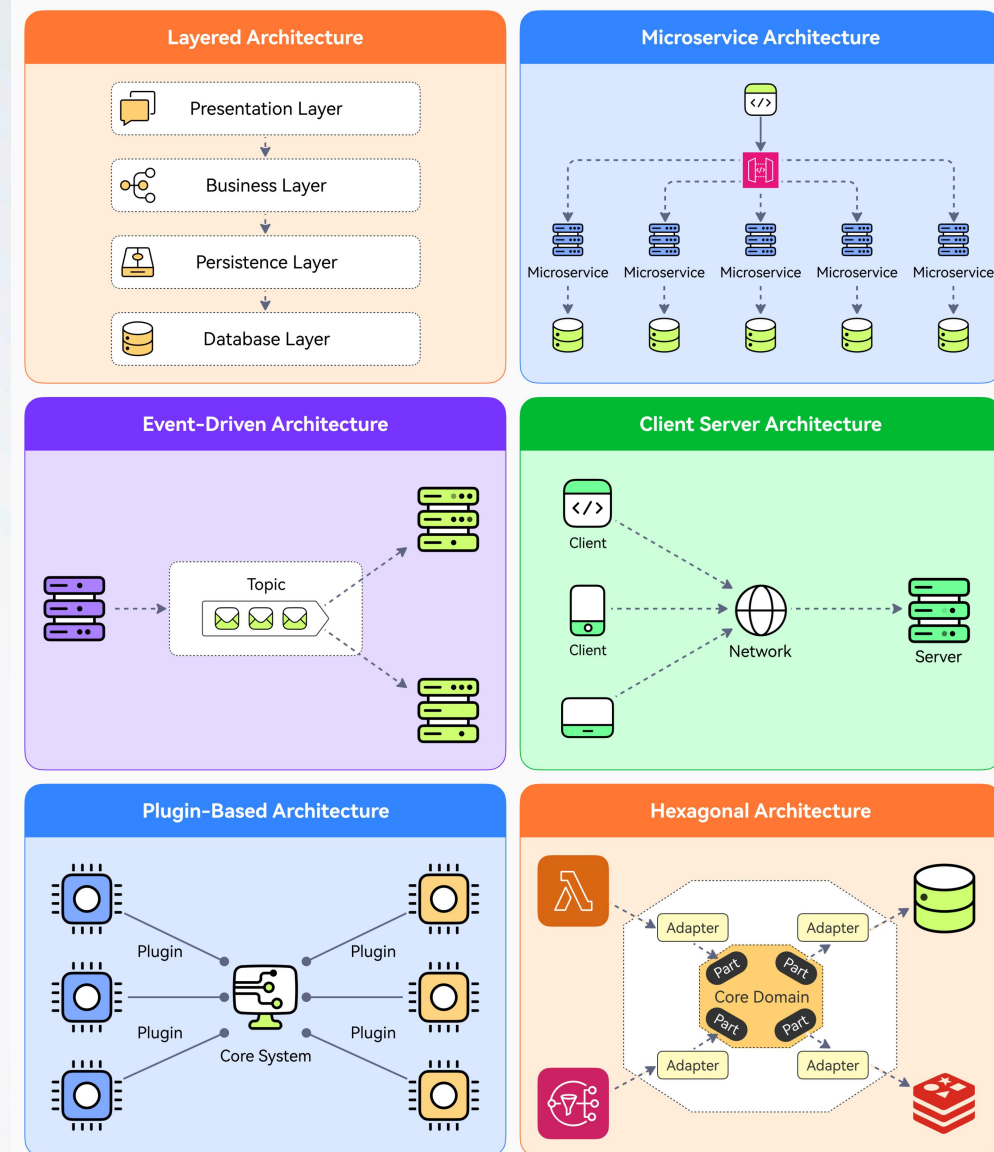
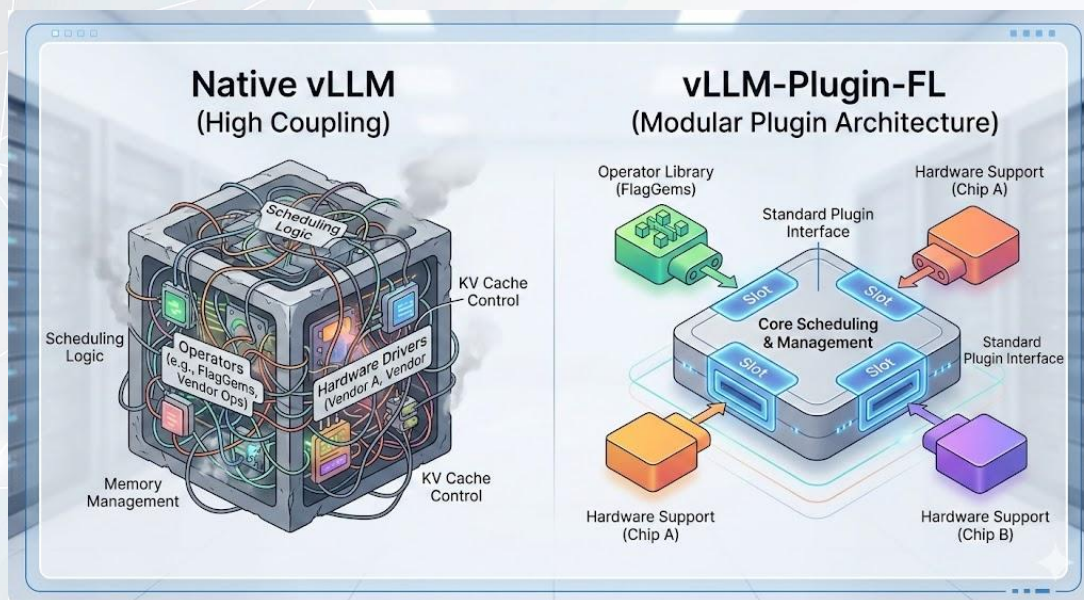
北京大学
PEKING UNIVERSITY

- What FlagOS does
 - Unified heterogeneous computing open-source software stack
 - "Write once, run on any hardware"
 - Supports NVIDIA, Days, Hygon, Huawei, Moore Threads



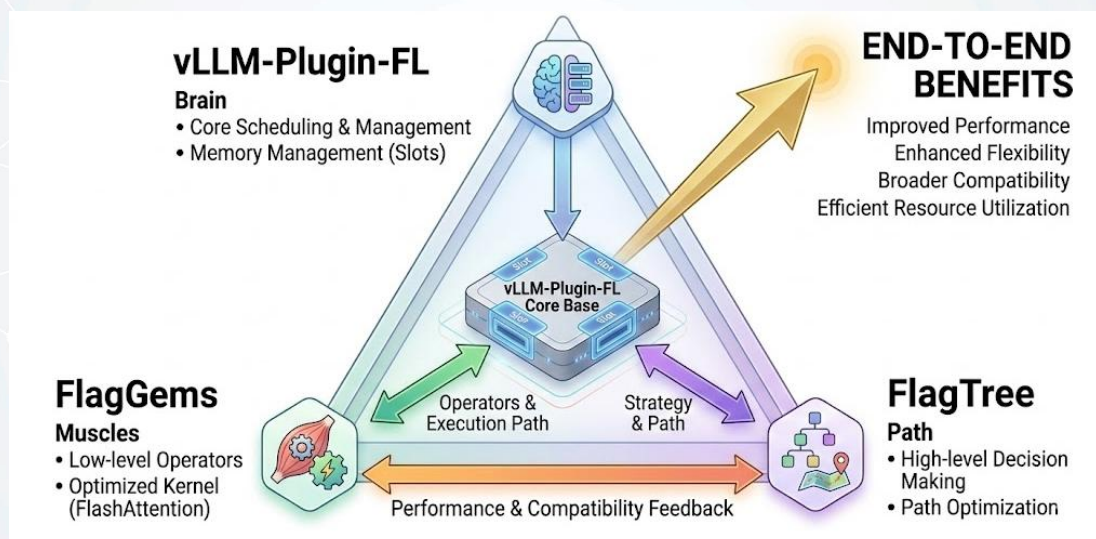
vLLM-Plugin-FL Design Philosophy

- vLLM-Plugin-FL's Plugin-Based Design:
 - Decoupling: Separating core scheduling logic from underlying hardware execution/operator libraries.
 - Hot-Swapping: Facilitating quick integration with self-developed optimization libraries such as FlagGems.



vLLM-Plugin-FL + FlagGems + FlagTree

- The Iron Triangle Collaboration Mechanism:
 - vLLM-Plugin-FL: Acts as the brain, responsible for efficient request scheduling, memory management (KV Cache), and computation graph distribution.
 - FlagGems: Acts as the muscles, providing highly optimized underlying kernels (such as FlashAttention and various fusion operators).
 - FlagTree: Provides higher-level optimization of decision trees or execution paths
- End-to-End Benefits: Achieves "out-of-the-box" high performance across multiple platforms.



- Benchmarking & Performance Metrics
- Profiling & Roofline Analysis
- Auto-Tuning & KernelGen
- End-to-End Optimization
- **Lab: Performance Analysis in Practice**

Module 5: Performance Evaluation & Optimization Practice



How to Run This Lab

- Lab 1 introduced how to write and tune a GEMM kernel.
- This module teaches you how to prove whether your changes actually help — scientifically, and layer by layer.
- **Repository**
 - Student repo: gitee.com/yihan-long/day2-lab5
 - Target platform: BI-V150 / FlagOS; SUT = Day2 GEMM (or the in-repo equivalent)

```
bash setup.sh && python3 00_smoke_cuda.py
export LAB_DAY2_ROOT=$PWD/../../lab1-day2

python3 01_benchmark.py --scenario offline
python3 01_benchmark.py --scenario server
python3 02_roofline.py
python3 03_profiling.py
python3 04_summary.py # then fill in REPORT.md
```

Environment smoke test → ① Benchmark → ②
Roofline → ③ Profiling → Fill in REPORT.md

Three Experiments – What Question Does Each One Answer?

Step	What We Do	What We Measure / Look At	The Question It Answers
① Benchmark	Run GEMM repeatedly through LoadGen (Offline / Server)	Median/P99 latency, throughput, effective TFLOP/s; compare torch / baseline / autotuned	How fast is it? How large is the gap?
② Roofline	Plot the benchmark results against hardware peak values	Arithmetic intensity (I), ridge point, position relative to the ceiling	Is the bottleneck compute or memory?
③ Profiling	Observe the same workload with ixsmi / ixsys / ixkn	Utilization, trace timeline, kernel-level evidence	Where exactly is it stalling? Does it agree with the Roofline diagnosis?



Thank you!